Laxmi Charitable Trust's

Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce

Department of Information Technology (B.Sc.I.T Semester V)

Dot .Net Core and Progressive Web Application Practical

Practical – V

Roll No.: T018	Name: Sachin Ramesh Mehta
Class: TYIT	Batch: 1
Date of Assignment: 21/07/2026	Date/Time of Submission: 28/07/2026

a. Create a web application to demonstrate the use of different types of Cookies.

Algorithm:

Step 1:- Start.
Step 2:- Display labels, textbox, and buttons.
Step 3:- Click View State button.
Step 4:- Store/Retrieve data using ViewState.
Step 5:- Display ViewState value.
Step 6:- Enter text in the textbox.
Step 7:- Click Cookie button.
Step 8:- Store textbox value in a cookie.
Step 9:- Retrieve and display the cookie value.
Step 10:- Stop.

Code:

Practical5 — Practical5.WebForm1

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

namespace Practical5
{
    2 references
    public partial class WebForm1 : System.Web.UI.Page
    {
        0 references
        protected void Page_Load(object sender, EventArgs e)
        {
            if(IsPostBack)
            {
                int ViewStateVal = Convert.ToInt32(ViewState["count"]) + 1;
                Label2.Text = "View State =" +ViewStateVal.ToString();
            }
            else
            {
                ViewState["count"] = 1;
            }
        }

        0 references
        protected void Button1_Click(object sender, EventArgs e)
        {
            Label1.Text= ViewState["count"].ToString();
        }

        0 references
        protected void Button2_Click(object sender, EventArgs e)
        {
            HttpCookie h = new HttpCookie("name");
            h.Value = TextBox1.Text;
            Response.Cookies.Add(h);
```

Label

Label

View State

Cokies

```
Practical5                                                    Practical5.WebForm2
 1   using System;
 2   using System.Collections.Generic;
 3   using System.Linq;
 4   using System.Web;
 5   using System.Web.UI;
 6   using System.Web.UI.WebControls;
 7
 8   namespace Practical5
 9   {
         1 reference
10       public partial class WebForm2 : System.Web.UI.Page
11       {
             0 references
12           protected void Page_Load(object sender, EventArgs e)
13           {
14               if (Request.Cookies["name"]!= null)
15               {
16                   Response.Write("Welcome" + Request.Cookies["name"].Value);
17               }
18           }
19       }
20   }
```

WebForm2.aspx | WebForm2.aspx.cs

body

Label

Output:

Label

View State =2

View State

Sachin

Cokies

Welcome : Sachin
Label

b. Create a Product Table in SQL Server and perform Insert Operation using Entity Framework Core.

Algorithm:

1. Start.
2. Create a database named **ProductDB**.
3. Create the **Product** model with **Id, Name, and Price**.
4. Create **AppDbContext** and add the **Products** table (DbSet<Product>).
5. Configure the SQL Server connection string in **appsettings.json**.
6. Register **AppDbContext** in **Program.cs**.
7. Create **HomeController** and inject **AppDbContext**.
8. Create a new product object and assign values (e.g., Laptop, 50000).
9. Add the product to the database using Add() and save it using SaveChanges().
10. Install Entity Framework Core packages.
11. Create and apply migrations using:

- Add-Migration InitialCreate
- Update-Database

12. Run the application.
13. Open **Home/Index** to insert the product.
14. Verify the inserted record in SQL Server using SELECT * FROM Products.
15. Stop.

Code:

Product.cs | SQLQuery1.sql * | ProductDB.sql

Practical-5

```
3 references
public class Product
{
    0 references
    public int Id { get; set; }
    1 reference
    public string Name { get; set; }

    1 reference
    public int Price { get; set; }
}
```

Practical-5 | HomeController

```
using Microsoft.AspNetCore.Mvc;
1 reference
public class HomeController : Controller
{
    private readonly AppDbContext _context;
    0 references
    public HomeController(AppDbContext context)
    {
        _context = context;
    }
    0 references
    public IActionResult Index()
    {
        Product p = new Product();
        p.Name = "Sachin's Laptop";
        p.Price = 50000;
        _context.Products.Add(p);
        _context.SaveChanges();
        return Content("Product Inserted Successfully");
    }
}
```

Schema: https://www.schemastore.org/appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection": "Server=(localdb)\\MSSQLLocalDB;Database=ProductDB;Trusted_Connection=True;TrustServerCertificate=True;"
  }

}
```

Output:

```
Product Inserted Successfully
```

```
select * FROM Products;
```

c. Develop a web application to perform CRUD Operations (Create, Read, Update, Delete) on Employee table using EF Core.

Algorithm:

1. Start.
2. Open Visual Studio and create an **ASP.NET Core MVC** project.
3. Create the **Employee** model with required properties.
4. Create the **AppDbContext** class.
5. Add DbSet<Employee> to the context.
6. Configure the connection string in **appsettings.json**.
7. Register AppDbContext in **Program.cs**.
8. Install Entity Framework Core packages.
9. Create and apply database migrations.
10. Create the **EmployeesController**.
11. Implement the **Create** operation to add employee records.
12. Implement the **Read** operation to display employee records.
13. Implement the **Edit** operation to update employee records.
14. Implement the **Delete** operation to remove employee records.
15. Run the application.
16. Test Create, Read, Edit, and Delete operations.
17. Verify the output.
18. Stop.

Code:

EmployeesController.cs:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using EmployeeCRUD.Models;
using EmployeeCRUD.Data;

1 reference
public class EmployeesController : Controller
{
    private readonly AppDbContext _context;

    0 references
    public EmployeesController(AppDbContext context)
    {
        _context = context;
    }

    // GET: EMPLOYEES
    3 references
    public async Task<IActionResult> Index()
    {
        return View(await _context.Employees.ToListAsync());
    }

    // GET: EMPLOYEES/Details/5
    0 references
    public async Task<IActionResult> Details(int? id)
    {
        if (id == null)
        {
```

Employee.cs:

```
namespace EmployeeCRUD.Models
{
    13 references
    public class Employee
    {
        11 references
        public int ID { get; set; }
        12 references
        public required string Name { get; set; }
        12 references
        public required string Department { get; set; }
        12 references
        public double Salary { get; set; }

    }
}
```

AppDbContext.cs:

```
using Microsoft.EntityFrameworkCore;
using EmployeeCRUD.Models;

namespace EmployeeCRUD.Data
{
    7 references
    public class AppDbContext:DbContext
    {
        0 references
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options){}
        7 references
        public DbSet<Employee> Employees { get; set; }
    }
}
```

Appsetting.json:

```
{
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "MyConnection": "Server=(localdb)\\MSSQLLocalDB;Database=EmployeeDB;Trusted_Connection=True;TrustServerCertificate=True",
    "AppDbContext": "Server=(localdb)\\mssqllocalDb;Database=AppDbContext;Trusted_Connection=True;MultipleActiveResultSets=true"
  }
}
```

Program.cs:

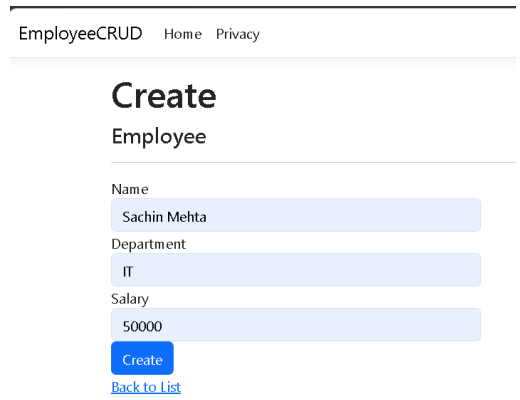
```
EmployeeCRUD | Program | <top-level-statements-entry-point>
using Microsoft.EntityFrameworkCore;
using EmployeeCRUD.Data;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<AppDbContext>(options => options.UseSqlServer(builder.Configuration.GetConnectionString("MyConnection")));

var app = builder.Build();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    // The default HSTS value is 30 days. You may want to change this for production scenarios, see https://aka.ms/aspnetcore-hsts.
    app.UseHsts();
}
```

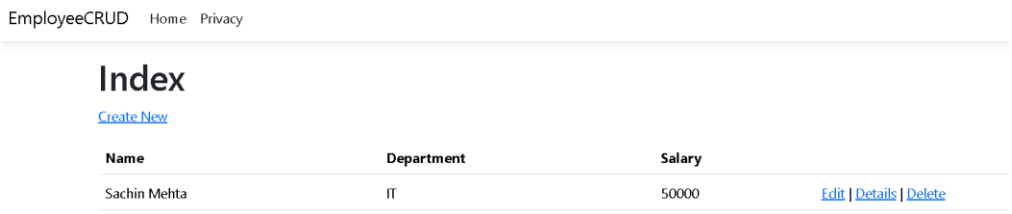
In **EmployeeCRUD.csproj** add this :

```
<NuGetAuditSuppress Include="https://github.com/advisories/GHSA-g4vj-cjjj-v7hg" />
```

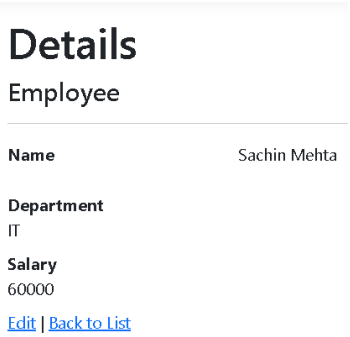
Output:

Create :-

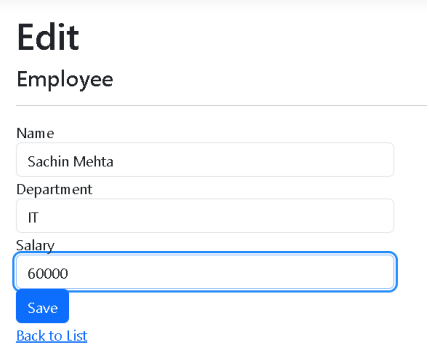




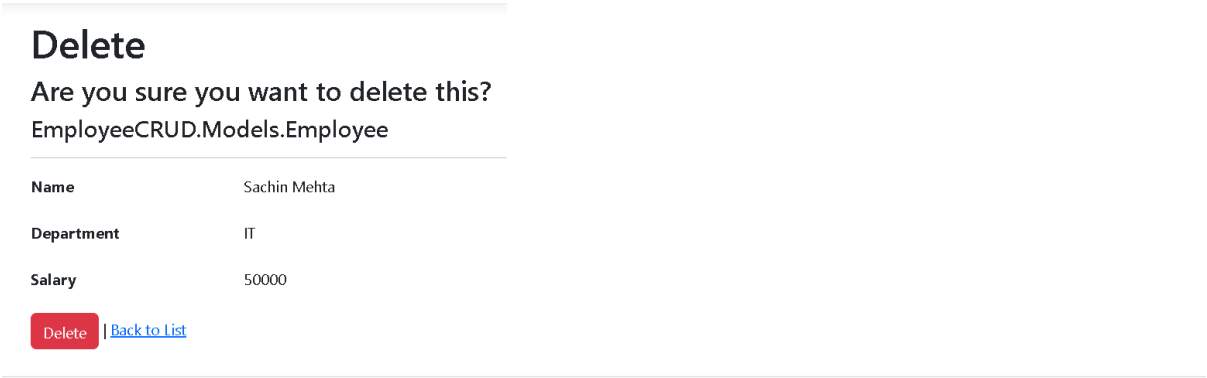
Read :-

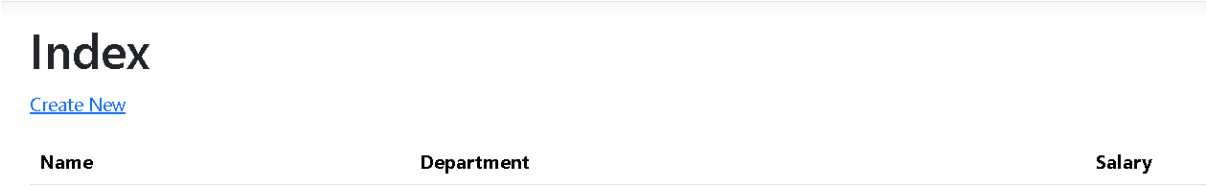


Edit :-



Delete :-





d. Authentication and Authorization in ASP.NET Core.

Algorithm:

1. Create an ASP.NET Core MVC project.
2. Enable Cookie Authentication in Program.cs.
3. Create Login action in HomeController.
4. Verify username and password.
5. Redirect to Secure page after successful login.
6. Use [Authorize] to protect the Secure page.
7. Logout and redirect to Login.

Code:

```
<h2>Login</h2>

<form method="post">

    Username :
    <input type="text" name="username" /><br /><br />

    Password :
    <input type="password" name="password" /><br /><br />

    <input type="submit" value="Login" />

</form>

<p style="color:red">
    @ViewBag.Message
</p>
```

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Authentication.Cookies;
using System.Security.Claims;

namespace AuthDemo.Controllers
{
    0 references
    public class HomeController : Controller
    {
        0 references
        public IActionResult Index()
        {
            return View();
        }

        [HttpPost]
        0 references
        public async Task<IActionResult> Index(string username, string password)
        {
            if (username == "Sachin" && password == "123")
            {
                var claims = new List<Claim>
                {
                    new Claim(ClaimTypes.Name, username)
                };

                var identity = new ClaimsIdentity(claims,
                    CookieAuthenticationDefaults.AuthenticationScheme);

                await HttpContext.SignInAsync(
                    CookieAuthenticationDefaults.AuthenticationScheme,
                    new ClaimsPrincipal(identity));

                return RedirectToAction("Secure");
            }
```

```
using Microsoft.AspNetCore.Authentication.Cookies;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
.AddCookie(options =>
{
    options.LoginPath = "/Home/Index";
});

builder.Services.AddAuthorization();

var app = builder.Build();

app.UseStaticFiles();

app.UseRouting();

app.UseAuthentication();
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

Output: